

Operating System, Section 2

Assignment3 - Simple scheduling Document

Yoo, J. H.

32212808

Department of Mobile Systems Engineering

2025-11-01

Table of Contents

1. 서론
2. 소프트웨어 요구사항 기술 (Requirements)
3. 소프트웨어 설계 (Design)
4. 소프트웨어 구현 (Implementation)
5. 시험 & 디버깅 (Verification)
6. 소프트웨어 유지보수 (Maintenance)
7. 결론

1. 서론

A. 개발 목적

본 프로젝트의 목적은 운영체제의 스케줄링 정책 중 하나인 라운드 로빈을 이해하고 프로그래밍을 통해 직접 구현하는 데 있다.

구체적으로 부모 프로세스가 여러 개의 자식 프로세스를 생성하고, 라운드 로빈 스케줄링 정책에 따라 프로세스에 CPU 시간을 분배하는 과정을 구현하고, 프로세스 간 통신과 타이머 인터럽트 처리 등 운영체제의 핵심 요소들을 실제 코드로 다뤄 봄으로써, 단순히 스케줄링 정책을 이해하는 것에서 나아가 실질적인 운영체제 동작 과정을 이해하는 것을 목표로 한다. 또한 프로세스가 CPU 작업과 I/O 작업을 반복하며 상태가 변화하는 실제 운영체제 환경을 보다 정확하게 모사함으로써 ready 큐와 wait 큐 관리, I/O 처리, time quantum 처리 등 스케줄러 동작 전반과 운영체제를 종합적으로 학습하는 것이 본 프로젝트의 핵심이다.

2. 소프트웨어 요구사항 기술 (Requirements)

A. 기능적 요구사항

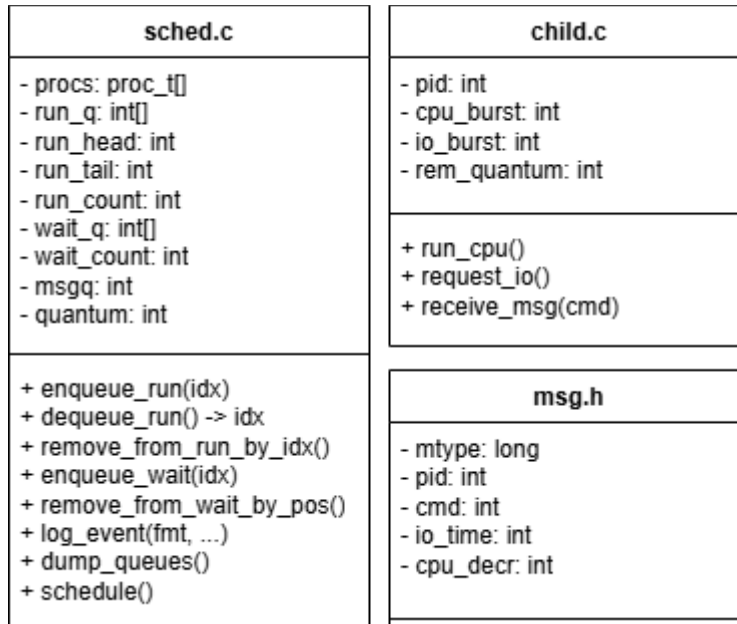
Makefile 작성
프로그램을 make 명령어로 컴파일할 수 있어야 한다.
프로세스 생성
부모 프로세스는 10 개의 자식 프로세스를 생성해야 한다.
라운드 로빈 스케줄링 정책
부모 프로세스는 자식 프로세스에 라운드 로빈 스케줄링 정책에 따라 CPU 시간을 분배해야 한다. Time quantum 은 인자로 받아야 한다.
타이머 인터럽트
부모 프로세스는 setitimer 시스템 콜을 통해 SIGALRM 인터럽트를 받아야 한다. 자식 프로세스는 CPU time slice 메시지를 수신하면 CPU burst 값을 감소시켜야 한다.
I/O 요청 & 상태 전환
자식 프로세스는 CPU burst 가 완료되면 부모 프로세스에게 I/O 요청 메시지를 송신해야 한다. 부모 프로세스는 자식 프로세스를 run 큐에서 wait 큐로 이동시키고, I/O-burst 값을 관리해야 한다. 자식 프로세스가 I/O 를 완료하면 wait 큐에서 run 큐로 이동시켜야 한다.
스케줄링 정보 출력
스케줄링 정보가 파일로 출력되어야 한다.

B. 비기능적 요구사항

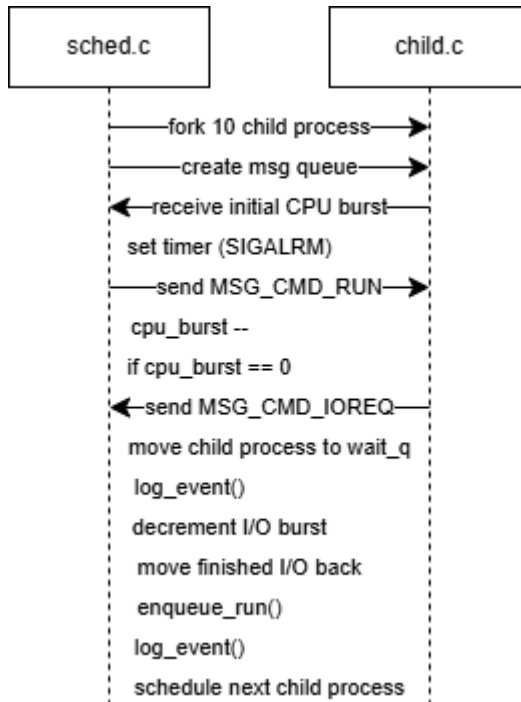
성능
스케줄링 성능이 최적화되는 Time quantum 을 제시해야 한다.
안정성
Deadlock 문제와 starvation 문제가 없어야 한다.
에러 처리
입력된 인자가 잘못된 경우, 오류 메시지를 출력해야 한다.
유지보수성
기능을 각각의 모듈로 분리해야 한다.
POSIX (Portable Operating System Interface + X)
POSIX 시스템 콜 사용을 준수해야 한다.

3. 소프트웨어 설계 (Design)

A. 아키텍처 설계 - 클래스 다이어그램



B. 아키텍처 설계 - 시퀀스 다이어그램



4. 소프트웨어 구현 (Implementation)

A. 개발 환경 & 도구

Operating System	
Ubuntu 22.04.5 LTS	
Kernel	
Linux 5.15.0-136-generic	
IDE (Integrated Development Environment) & Editor	
vi	
Programming language	
C	
Framework	
errno.h	Error library
signal.h	Signal handling library
stdarg.h	Variable arguments handling library
stdio.h	Standard input & output library
stdlib.h	Standard library
string.h	String library
sys/msg.h	System V message queue library
sys/time.h	Time value and interval library
sys/types.h	System data types library
sys/wait.h	Process wait handling library
time.h	Time library
unistd.h	POSIX API
Build tool	
GCC compiler with Makefile	

B. 모듈별 기능 설명

msg.h
부모 프로세스와 자식 프로세스 간의 메시지 전달을 위한 구조체다.
sched.c
부모 프로세스 실행 파일이다. 10 개의 자식 프로세스를 생성하고 라운드 로빈 스케줄링 정책을 처리한다. 구체적으로 CPU 시간 분배, 타이머 인터럽트 처리, run 큐와 wait 큐 관리를 수행한다.
child.c
자식 프로세스 실행 파일이다. CPU burst 실행, I/O 요청 메시지 송신, I/O burst 실행을 반복한다.

C. 주요 알고리즘 설명 & Code Snippet

```
// System V message buffer
struct msgbuf_s{
    long mtype;      // Message type (parent process to child
process message is PID, child process to parent process message
is 1)
    pid_t pid;      // Sender process PID
    int cmd;        // Command (MSG_CMD_RUN or MSG_CMD_IOREQ)
    int io_time;    // If command is MSG_CMD_IOREQ then use io_time
for request I/O time
    int cpu_decr;   // How many cpu ticks child progressed
};
```

부모 프로세스와 자식 프로세스 간 통신의 구현 방식은 이전 과제인 Multi-thread word count 에서 생산자 스레드와 소비자 스레드 간 통신 구현 방식과 매우 유사하다.

생산자 스레드와 소비자 스레드 간 통신에 공유 버퍼를 이용한 것처럼 부모 프로세스와 자식 프로세스 간 통신은 System V message queue 구조체를 이용한다.

프로세스를 실행해도 될 경우 부모 프로세스가 자식 프로세스로 MSG_CMD_RUN 메시지를 송신한다.

CPU burst 가 종료된 경우 자식 프로세스는 부모 프로세스로 MSG_CMD_IOREQ 메시지를 송신한다.

```

if(rcv.cmd == MSG_CMD_IOREQ){
    int idx = -1;
    for(int k = 0; k < NUM_CHILD; k++){
        if(procs[k].pid == rcv.pid){
            idx = k;
            break;
        }
    }

    if(idx == -1){
        continue;
    }

    // Move child process to wait queue
    remove_from_run_by_idx(idx);
    procs[idx].io_burst = rcv.io_time;
    procs[idx].in_wait = 1;
    enqueue_wait(idx);

    // Log
    log_event("----- Time tick %d -----\n • Process %d
requests I/O and moved to wait-queue\n", current_tick, idx);
    dump_queues(current_tick);

    if(running_idx == idx){
        running_idx = -1;
    }
}

```

부모 프로세스는 System V message buffer 의 mtype 변수를 통해 자식 프로세스가 I/O 요청을 했는 지 판단한다.

I/O 요청이 온 경우 부모 프로세스는 자식 프로세스를 wait 큐로 이동시킨다.

I/O burst 는 time tick 마다 감소하고, I/O burst 가 완료된 경우 부모 프로세스는 자식 프로세스를 run 큐로 이동시킨다.


```
// ----- Main child loop -----
while(1){
    // 생략

    // If parent allows to run
    if(rcv.cmd == MSG_CMD_RUN){
        cpu_burst--; // Decrease CPU burst by 1 tick

        // If CPU burst is finished then request I/O
        if(cpu_burst <= 0){
            // Generate I/O burst time (10 ~ ticks)
            int next_io = (rand() % 91) + 10;

            // Send I/O burst to parent process
            memset(&snd, 0, sizeof(snd));
            snd.mtype = 1;
            snd.cmd = MSG_CMD_IOREQ;
            snd.pid = getpid();
            snd.io_time = next_io;
            if(msgsnd(msgq, &snd, sizeof(snd) - sizeof(long), 0)
== -1){
                perror("child msgsnd ioreq");
            }

            // Generate new CPU burst time (10 ~ ticks)
            cpu_burst = (rand() % 91) + 10;
        }
    }
}
```

자식 프로세스는 부모 프로세스로부터 MSG_CMD_RUN 메시지를 수신 받는 동안 CPU burst 를 1tick 씩 감소시킨다.

자식 프로세스는 CPU burst 가 0 이 된 경우 I/O burst 시간을 난수로 생성하고, 부모 프로세스에게 MSG_CMD_IOREQ 메시지를 전송한다.

자식 프로세스는 새로운 CPU burst 시간을 난수로 생성하고 위 과정을 반복한다.

```
// Dump run queue and wait queue state
void dump_queues(){
    fprintf(logfp, "• Run queue (Index, Remaining CPU burst,
Remaining time quantum)\n[");
    for(int i = 0; i < run_count; i++){
        int idx = run_q[(run_head + i) % NUM_CHILD];
        fprintf(logfp, " (%d, %d, %d)%s", idx,
procs[idx].cpu_burst, procs[idx].rem_quantum, (i == run_count -
1) ? "" : ", ");
    }
    fprintf(logfp, "]\n• Wait queue (Index, Remaining I/O burst,
Remaining time quantum)\n[");
    for(int i = 0; i < wait_count; i++){
        int idx = wait_q[i];
        fprintf(logfp, " (%d, %d, %d)%s", idx,
procs[idx].io_burst, procs[idx].rem_quantum, (i == wait_count -
1) ? "" : ", ");
    }
    fprintf(logfp, "]\n");
    fflush(logfp);
}
}
```

Ready 상태인 자식 프로세스는 run 큐에서 관리한다.

Run 큐는 처음에는 선형 큐로 구현하였으나 현재는 원형 큐로 구현되어 있다.

Run 큐를 선형 큐로 구현할 경우, 큐의 앞에 있는 프로세스가 ready 큐로 이동해 빈 경우에도 큐가 꽉 찬 것처럼 동작해서, 빈 공간이 있음에도 overflow 에러가 발생하는 문제가 있었다.

I/O wait 상태인 자식 프로세스는 wait 큐에서 관리한다.

Time tick 마다 Run 상태인 자식 프로세스의 CPU burst 처리 작업, 자식 프로세스가 time quantum 을 소진한 경우 run 큐의 뒤로 이동하는 작업, 자식 프로세스가 I/O 작업을 완료한 경우 run 큐로 이동하는 작업을 모두 로그로 출력한다.

5. 시험 & 디버깅 (Verification)

시험 목록	입력	출력	상태
Makefile 빌드	make	.o 파일 생성	Pass
Makefile 클리어	make clean	.o 파일 삭제	Pass
인자 없음	./sched	• ./sched [time_quantum]	Pass
잘못된 인자	./sched -10	• Time quantum must be positive integer	Pass
time_quantum 인자	./sched 10	• Message queue id is 32768, log.txt 파일 생성	Pass

소프트웨어 요구사항 기술의 비기능적 요구사항 중 하나인 스케줄링 성능이 최적화되는 Time quantum 을 제시하기 위한 시험을 진행하였다.

구체적인 시험 방식은 아래와 같다.

1	Simple scheduling 의 time_quantum 인자를 10, 20, 40, 80, 160 로 시행한다.
2	각 시행으로 생성된 log.txt 파일을 분석한다. 구체적으로 log.txt 파일의 '• Process N gets CPU' 로그와 '• Process N time quantum expired' 로그의 개수를 구한다. 이 로그의 개수가 많을수록 context switch 가 자주 발생하고 오버헤드가 증가함을 의미한다. log.txt 파일의 run 큐의 평균 크기를 구한다. Run 큐의 평균 크기가 클수록 프로세스의 평균 대기 시간이 깊을 의미한다.
3	로그의 개수가 적고, run 큐의 평균 크기가 작은 time_quantum 을 구한다.

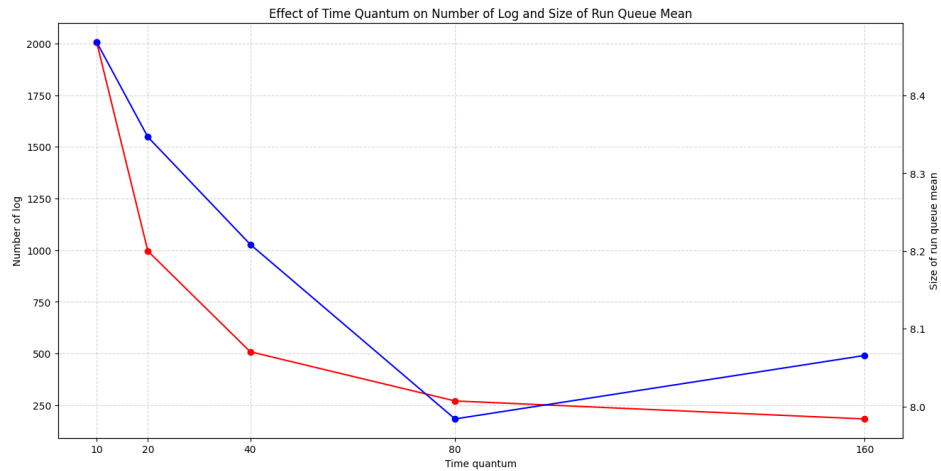
Log.txt 파일의 분석은 test.py 파일을 제작하여 사용하였다.

구체적인 시험 결과는 아래와 같다.

Time quantum	로그의 개수	Run 큐의 평균 크기
10	$1078 + 929 = 2007$	8.468830059777968
20	$587 + 409 = 996$	8.346686088856519
40	$343 + 166 = 509$	8.20852534562212
80	$223 + 48 = 271$	7.983948635634029
160	$184 + 0 = 184$	8.065573770491802

본 프로젝트의 최소 CPU burst 시간은 10 에서 100 사이이고, I/O burst 시간은 10 에서 100 사이로, 이 값이 변경되면 결과가 달라질 수 있다.

구체적인 시험 결과를 graph.py 파일을 제작하여 아래 그래프로 나타내었다.



로그의 개수는 빨간 선으로 낮을수록 성능이 좋음을 의미한다.

로그의 개수 관점에서는 time quantum 이 클수록 성능이 좋다는 것을 알 수 있다.

Run 큐의 평균 크기는 파란 선으로 낮을수록 성능이 좋음을 의미한다.

Run 큐의 평균 크기는 time quantum 이 다시 커지는 것을 확인할 수 있는데, 이는 최대 CPU burst 시간이 100 이기 때문인 것으로 보인다.

따라서 time quantum 은 10, 20, 40, 80, 160 중 80 이 성능이 가장 좋다고 할 수 있다.

6. 소프트웨어 유지보수 (Maintenance)

모듈화 설계 & 확장성
기능을 각각의 모듈로 분리하였다. 단일 책임 원칙 (SRP, Single Responsibility Principle)을 준수하여 코드 수정 시 다른 모듈에 대한 영향을 최소화했다.
에러 처리
잘못된 인자가 입력된 경우 사용자에게 오류 메시지를 출력하도록 하였다. fork(), exec(), msgget(), msgrcv(), msgsnd(), sigaction() 등의 시스템 콜 실행을 실패한 경우 오류 메시지를 출력하고 종료하도록 하였다.
코드 가독성
모든 모듈과 함수에 대한 주석을 작성하여 코드 이해도를 높였다.

7. 결론

본 프로젝트를 통해 sys/msg 등의 처음 사용해보는 프레임워크들을 다루며 새로운 시스템 구성 방식을 알게 되었다. 하지만 프레임워크의 함수들의 사용 방법을 익히는 것이 어려웠고, run 큐를 원형 큐로 변경하면서 디버깅 과정에서 많은 시행착오를 겪었다. 또한 스케줄링 프로그램을 한 번 실행하는 데 긴 시간이 걸려서 다양한 시험을 진행하지 못한 점이 부족한 점이자 개선할 점이라고 생각한다.

본 프로젝트를 통해 운영체제 개발자들은 단순히 프로그램을 완성시키는 것으로 끝나지 않고, 결과를 분석하고 성능을 측정하는 과정까지 하는 것을 대단하다고 느꼈다. 그리고 그런 개발자가 되기 위해서는 결과 분석과 성능 측정이 중요하다고 느꼈다. 앞으로 시스템 프로그래밍이나 운영체제 관련 프로젝트를 수행할 때 이러한 경험이 도움이 될 것이라고 생각한다.